Given a string `s`, return *the number of* ***palindromic substrings*** *in it*.

A string is a **palindrome** when it reads the same backward as forward.

A **substring** is a contiguous sequence of characters within the string.

Example 1:

```
Input: s = "abc"
Output: 3
Explanation: Three palindromic strings: "a", "b", "c".
```

Example 2:

```
Input: s = "aaa"
Output: 6
Explanation: Six palindromic strings: "a", "a", "a", "aa", "aa", "aaa".
```

String s = "abc"

Pal ⇒ reverse of string is equal to string

substring ⇒ a, b, c, ab, bc, ~~ac~~, abc. → 3 ⇒ Ans.

Recursion ⇒ choices + decisions. {a} ↺ {palindrome}

0 1 2 3 4
n i t i n

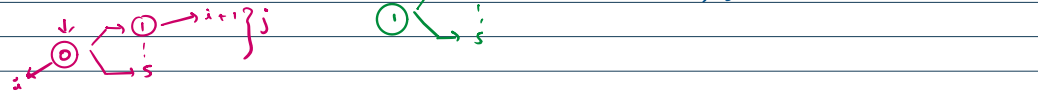
0-1 n
0-2 n i
0-3 n i t
0-4 n i t i
0-5 n i t i n

1-2 i
1-3 i t
1-4 i t i
1-5 i t i n

2-3 t
2-4 t i
2-5 t i n

3-4 i
3-5 i n

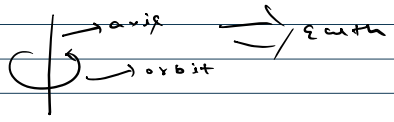
4-5 n



```
for(int i=0; i<s.len; i++) {
    for(int j=i+1; j<=s.len; j++) {
        sop(i + " " + j);
    }
}
```

Expand around center

Axis orbit approach



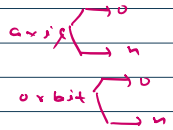
axis-orbit
axis+orbit
orbit = 1

n i t i n
t ⇒ pal.
axis
i ⇒ axis-orbit
i ⇒ axis+orbit
} equal ⇒ pal.

1. Count all pal. sub
2. Find longest pal. sub
3. Count all odd-length pal. sub
4. " " even "
5. Find longest odd-length pal. sub
6. " " even "

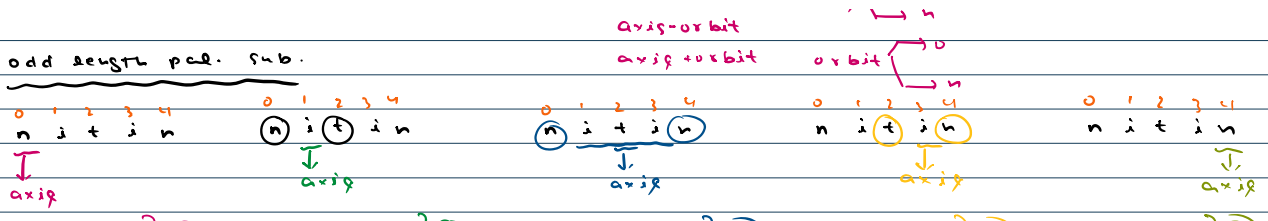
odd ⇒ a, aba, ababa ⇒ pal.
Even ⇒ aa, baab, cbaab ⇒ pal.

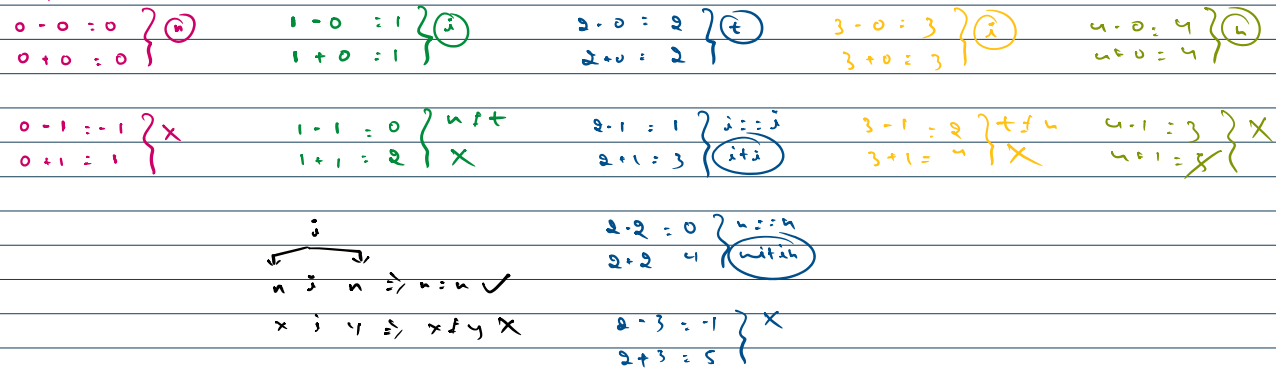
odd length pal. sub.



0 1 2 3 4 . . . 2 3 4 0 1 2 3 4 0 1 2 3 4

Odd length pal. sub.



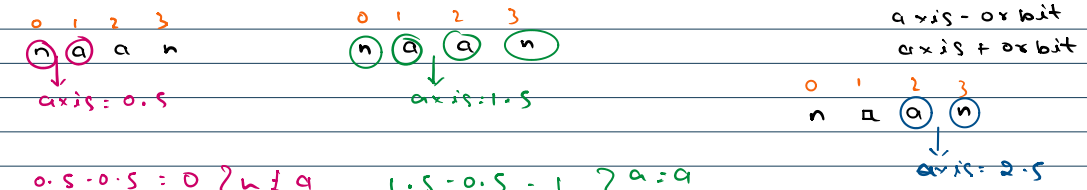


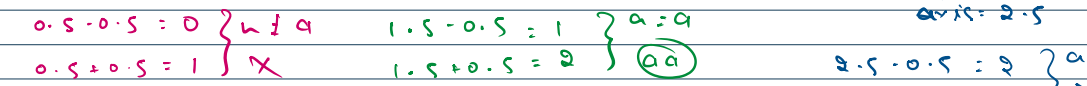
orbit out of bound ⇒ axis change.

or

char mismatch

Even-length pal. substring





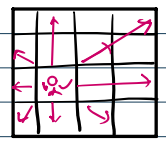
char mismatch

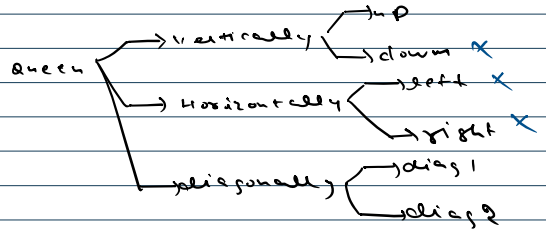
or

orbit out of

index ⇒ axis change ~~increment~~

N Queens



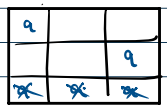


for, n = 1

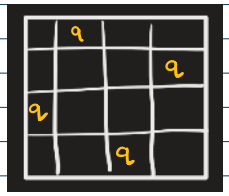
for, n = 2

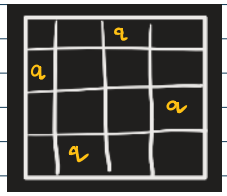
for, n = 3

For, n = 3

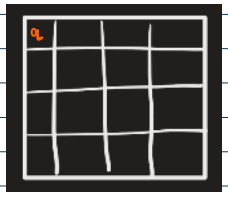


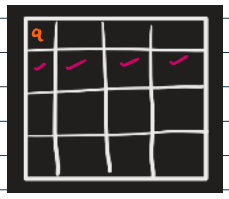
For, n = 4





only need to check up, diagonally right up, diagonally left up.

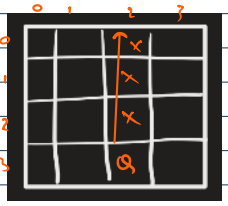




Is it safe or not

- up
- diagonally left
- diagonally right

up

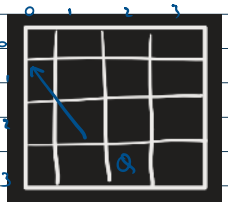


(3,2) : placed

(2,2)
(1,2)
(0,2)

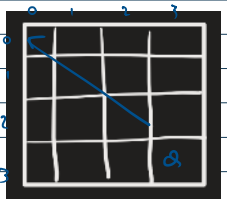
```
while(r>=0) {
    if(board[r][col]==true) {
        return false
    }
    r--;
}
```

Left diagonal.



placed : (3,2)

(2,1)
(1,0)



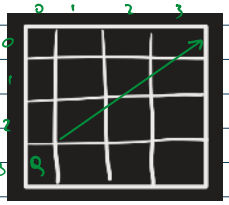
placed : (3,3)

(2,2)
(1,1)
(0,0)

```
r = row
c = col
while(r>=0 && c>=0) {
    if(board[r][c]==true) {
        return false
    }
    r--;
    c--;
}
```

right diagonal

right diagonal



```
private static void nQueensSol(boolean[][] board, int n, int row) {
    for (int col = 0; col < board[0].length; col++) {
        if (isSafe(board, row, col)) {
            board[row][col] = true;
            nQueensSol(board, n, row + 1);
        }
    }
}
```